AIコード生成の非決定性

再現性リスクと運用上の留意点

(株) SRA

2026-04-30

1. AIコード生成は再現できるとは限らない

- 生成AIによるコード生成は有効な開発支援手段
- ただし、同じ指示でも同じ結果になるとは限らない
- 大規模・自動化・本番適用では、品質ばらつきがリスクになる
- 「生成させること」より「検証できること」が重要

2. AIコード生成は決定的な処理ではない

生成AIサービスの出力について、同一入力から常に同一コードが得られると前提にしないことが重要です。結果はモデル、設定、入力、実行基盤、ツールの状態などに依存します。

- 同じプロンプトでも別の実装が出ることがある
- 少しの表現差でコード品質が変わることがある
- モデルやサービス側の更新で結果が変わることがある
- もっともらしいが誤ったコードが出ることがある

3. ばらつきを抑えても完全には固定できない

AIの回答ができるだけ安定する設定にしても、毎回同じコードが生成されるとは限りません。

- 生成AIサービスやAPIによっては、回答のばらつきを抑える設定が用意されている
- ただし、この種の設定を使っても、毎回同じ結果になるとは限らない

4. よくある誤解

誤解	実際
ばらつきを抑えれば毎回同じ	完全な再現性は保証されない
動いたコードなら安全	境界条件・保守性・脆弱性は別問題
AI生成コードはレビューを減らせる	むしろレビュー観点が増える場合がある
小規模で便利なら大規模でも同じ	大規模では技術的負債が蓄積しやすい

5. 非決定性が起きる主な要因

- モデル推論の確率的な性質
- プロンプトや入力コンテキストのわずかな違い
- モデル、API、推論基盤の更新
- 使用ライブラリや実行環境の違い
- エージェント型ツールによる探索経路の違い

6. 非決定性そのものが常に悪いわけではない

- 同じ仕様から複数の実装が生まれること自体は自然
- 最終的なコードが仕様を満たし、検証できるなら問題は小さい
- 仕様・テスト・レビューが弱い場合は品質ばらつきが見えにくい
- 問題は「違うコードが出ること」ではなく「違いを評価できないこと」

7.「テストを通ればよい」という立場

- 人間が書くコードも、同じ仕様から同じ実装になるとは限らない
- AI生成コードの非決定性も、実装差分の一種と考えられる
- ブラックボックステストで外部仕様、ホワイトボックステストで内部構造を確認する
- 両方を十分に満たせば受け入れ可能、という立場がある
- ただし、保守性・設計一貫性・セキュリティまで検証できることが前提

8. 実務で起きるリスク

- 同じ作業指示から異なるコード変更が作られる
- テストは通るが、設計方針と合わない実装が混入する
- エラー処理、入力検証、実行権限確認が抜ける
- レビュー対象が増え、確認コストが上がる
- 後から「なぜこのコードになったか」を説明しにくい

9. 大規模適用で特に注意すべき点

小さな補助では便利でも、大量適用ではリスクが増える。

- 生成コードの品質ばらつきが累積する
- 設計・命名・実装方針が揺れる
- レビューで見落とした問題が長期間残る
- セキュリティ弱点や保守性問題が混入する
- 自動化すると人間の判断点が見えにくくなる

10. 安全に使いやすい場面

- 既存コードの説明
- テストケースのたたき台作成
- 小さな関数や定型処理の補助
- エラーメッセージの調査
- リファクタリング案の比較
- 人間が理解・確認できる範囲の作業

11. 注意が必要な場面

- 認証・認可・課金・個人情報に関わる処理
- 暗号、検証方法、SQL 文、外部通信などの重要ロジック
- 大量ファイルの一括修正での漏れや誤修正
- 仕様が曖昧なままの実装依頼 (結果のぶれが大きい)
- 生成結果をそのまま本番投入する運用 (危険性が高い)
- 生成結果の根拠や変更理由を記録しない運用

12. 推奨する運用方針

- AI生成コードも通常コードと同じ品質基準で扱う
- 生成時のプロンプト、モデル、設定、出力を記録する
- 小さな変更単位に分けて適用する (精度の高いレビューがしやすい)
- テスト、静的解析、セキュリティチェックを必須にする
- 重要領域では人間レビューを省略しない
- 採用理由と却下理由を残す (後から振り返るため)

13. 推奨する使い方: AIは候補生成にとどめる

AIの非決定性を、設計や品質の責任から切り離す。

- AIが異なるコードを出すこと自体は許容する
- 採用するコードは、人間のレビュー・テスト・設計判断を通したものに限定
- 初期実装では、AIを実装候補の作成に使う
- 受け入れ後のコードは、通常の保守対象として管理
- 仕様から本番コードまでの全自動生成は避ける (リスクが高すぎる)

14. 導入時のチェックリスト

- 利用してよい業務範囲を決めているか
- 機密情報や顧客情報を入力しないルールがあるか
- 生成コードのレビュー責任者が明確か
- テストと静的解析がCIに組み込まれているか
- 生成過程と採用判断を後から追跡できるか
- 本番投入前の承認フローがあるか

15. 結論

生成AIコード生成は、使い方次第で開発効率を高める。 出力は常に安定しているとは限らない。 利用する場合は、次の前提が重要。

- AIは「実装候補を出す道具」
- 品質は「検証プロセス」で担保
- 大規模適用では「記録・レビュー・監査」が必要

16. 参考研究

- Ouyang, Zhang, Harman, Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation", ACM TOSEM, 2025 研究で対象としたモデル・課題・条件で、出力の意味・構文・構造の揺れを調べた。現在の全モデル・全課題の品質を示すものではない。 ばらつきを抑える設定でも完全な決定性は保証されない点を示す。 https://research-information.bris.ac.uk/en/publications/an-empirical-study-of-the-non-determinism-of-chatgpt-in-code-gene

- Mastropaolo ほか, "On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot", 2023 意味が同じでも、自然言語プロンプトの言い換えで生成コードが変わることを調査。 プロンプト表現の揺れが、コードの正しさや品質に影響する点を論じる。 https://arxiv.org/abs/2302.00438

- Liu ほか, "Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild", arXiv, 2026 GitHub 上の AI 生成コードを大規模に分析し、コードの不吉な臭いや正しさの問題を追跡。 AI 生成コードが技術的負債として残る可能性を論じる。 https://arxiv.org/abs/2603.28592

- Fu ほか, "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study", 2023 実プロジェクトに含まれる AI 生成コード片を対象に、セキュリティ弱点を分析。 入力検証、コード実行、乱数利用などの脆弱性混入リスクを示す。 https://arxiv.org/abs/2310.02059

- Pearce ほか, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions", 2021 高リスクなセキュリティシナリオで Copilot にコードを生成させ、脆弱性の有無を評価。 生成結果をそのまま信頼せず、セキュリティレビューが必要であることを示す。 https://arxiv.org/abs/2108.09293

17. 出力のばらつきを調整する設定

- 出力のばらつきを調整できる設定の有無は、サービスだけでなく、利用するモデルとAPIによって異なる。対象の公式仕様で、パラメーターの対応と制約を確認する。 ただし、低い値を指定しても、各サービスの仕様上、完全な再現性が保証されるとは限らない。

  参照:
  https://developers.openai.com/api/reference/resources/responses/methods/create
  https://learn.microsoft.com/en-us/azure/foundry/openai/reference
  https://docs.anthropic.com/en/api/messages
  https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/content-generation-parameters